Remote Access Gateway Configuration & Traffic Control

Remote Access Gateway Configuration & Traffic Control is the process of configuring a secure gateway that enables authenticated remote users to access organizational resources while monitoring and controlling network traffic. It enforces security policies such as authentication, authorization, encryption, firewall rules, and access control to ensure that only legitimate users and permitted traffic can enter the network. This approach protects enterprise systems from unauthorized access, malware, and network-based attacks while maintaining secure remote connectivity.

Introduction

As organizations increasingly support remote work, secure access to internal resources has become essential. A **Remote Access Gateway** acts as a secure entry point between remote users and the organization's internal network. It authenticates users, encrypts communication, and controls network traffic based on predefined security policies. Traffic control mechanisms ensure that only authorized users and approved network traffic are allowed, reducing the risk of cyberattacks and unauthorized access.

Objectives

The objective of **Remote Access Gateway Configuration & Traffic Control** is to provide secure remote access to organizational resources while monitoring and controlling network traffic. It ensures that only authenticated users and authorized devices can connect to the network through encrypted communication. The system enforces security policies, blocks unauthorized traffic, and protects enterprise resources from cyber threats. It also improves network security, availability, and overall access management..

Requirements

- A server (Linux or Windows) configured as a **Remote Access Gateway**.
- Client devices (Windows/Linux) for remote access testing.
- VPN software (e.g., OpenVPN or WireGuard) or SSH for secure remote connectivity.
- Firewall (UFW, iptables, or Windows Firewall) for traffic filtering.
- Stable network connectivity between the gateway and clients.
- Administrative privileges to configure gateway services and security policies.
- Network monitoring tools (e.g., Wireshark or tcpdump) for traffic analysis.
- Internet or LAN access to test secure remote connectivity and traffic control.

Step 1: Configure Virtual Machines

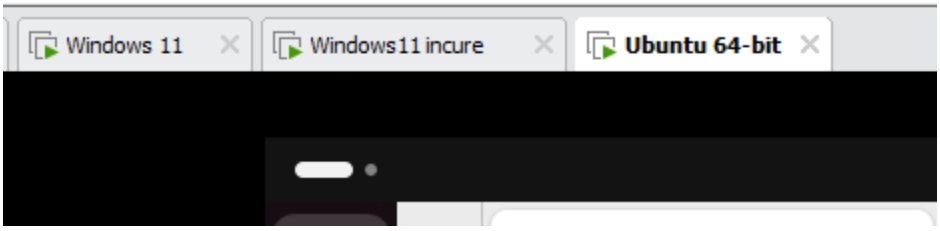

Fig: 2.1 Virtual Machines

Step 2: Install OpenSSH Server

On Ubuntu Server, install the SSH service.

Command:

sudo apt update

sudo apt install openssh-server -y

Verify the SSH service.

Command:

sudo systemctl status ssh

```
yagnesh@yagnesh:~$ sudo apt update
[sudo] password for yagnesh:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
yagnesh@yagnesh:~$ sudo apt install openssh-server -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  libfwupd2
```

Fig: 2.2 Installation of ssh

Step 3: Install Apache Web Server

Install Apache to expose HTTP service on port 80.

Command:

sudo apt install apache2 -y

Command:

sudo systemctl status apache2

```
yagnesh@yagnesh:~$ sudo apt install apache2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
apache2 is already the newest version (2.4.58-1ubuntu8.15).
The following package was automatically installed and is no longer required:
  libfwupd2
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
yagnesh@yagnesh:~$
```

Fig 2.3 Installing Apache

Step 4: Verify ssh services

sudo systemctl status ssh
sudo systemctl enable ssh
sudo systemctl restart ssh
Expected Output:
Active: active (running)

Step 6: Verify ubuntu ip

ip a
Verify the Host-Only IP address.
Example:
192.168.30.128

Step 7: Create remote user

sudo adduser remoteuser
Set a password when prompted.

```
yagnesh@yagnesh:~$ sudo adduser remoteuser
[sudo] password for yagnesh:
info: Adding user `remoteuser' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `remoteuser' (1001) ...
info: Adding new user `remoteuser' (1001) with group `remoteuser (1001)' ...
info: Creating home directory `/home/remoteuser' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password fails the dictionary check - it does not contain enough DIFFERENT character
Retype new password:
Sorry, passwords do not match.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
Sorry, passwords do not match.
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for remoteuser
Enter the new value, or press ENTER for the default
        Full Name []:
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n] y
info: Adding new user `remoteuser' to supplemental / extra groups `users' ...
info: Adding user `remoteuser' to group `users' ...
```

Fig: 2.4 create user

Step 8: Show Created user

cat /etc/passwd

Verify that the following user exists:

remoteuser:x:1001:1001:/home/remoteuser:/bin/bash

```
yagnesh@yagnesh:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:996:996:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
messagebus:x:101:101::/nonexistent:/usr/sbin/nologin
syslog:x:102:102::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:991:991:systemd Resolver:/:/usr/sbin/nologin
```

Fig: 2.5 Show user

Step 9: Configure SSH file

```
  GNU nano 7.2                          /etc/ssh/sshd_config *
#PermitTunnel no
#ChrootDirectory none
#VersionAddendum none

# no default banner path
#Banner none

# Allow client to pass locale environment variables
AcceptEnv LANG LC_*

# override default of no subsystems
Subsystem       sftp    /usr/lib/openssh/sftp-server

# Example of overriding settings on a per-user basis
#Match User anoncvs
#       X11Forwarding no
#       AllowTcpForwarding no
#       PermitTTY no
#       ForceCommand cvs server
PasswordAuthentication yes

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo
```

Fig 2.6 Configure ssh file

Step 10: Test ssh connection

Test ssh connection

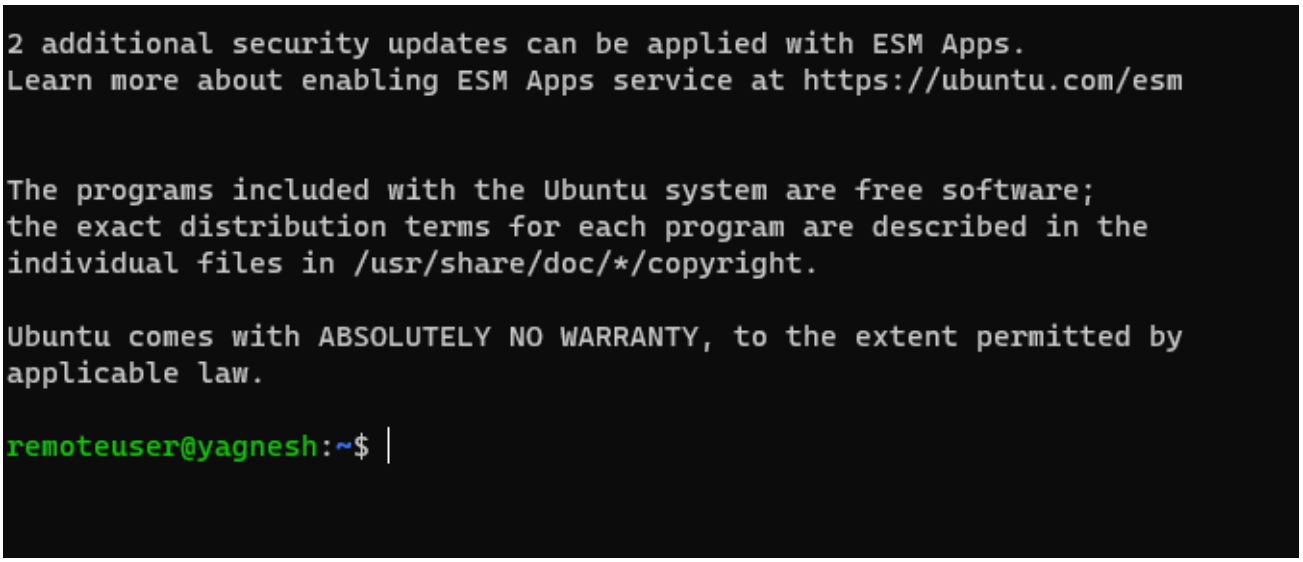

Fig : 2.7 Test ssh connection

Step10: Clear Existing rule

sudo iptables -F
sudo iptables -X

Step11: Configure Firewall Rules

Allow loopback traffic.
sudo iptables -A INPUT -i lo -j ACCEPT
Allow established connections.
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
Allow SSH.
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
Allow HTTP.
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
Allow ICMP (Ping).
sudo iptables -A INPUT -p icmp -j ACCEPT
Drop all remaining incoming traffic.
sudo iptables -P INPUT DROP

Step 12: Verify Firewall Rules

sudo iptables -L -n
Expected Output:
INPUT (policy DROP)

ACCEPT tcp dpt:22
ACCEPT tcp dpt:80
ACCEPT icmp

Step 13: Start Apache Web Server

sudo systemctl start apache2
sudo systemctl enable apache2
Verify status.
sudo systemctl status apache2

Step 15: Block Unauthorized Client

Assume Windows Client 2 has IP address:
192.168.56.30
Block the client.
sudo iptables -A INPUT -s 192.168.56.30 -j DROP
Verify firewall rules.
sudo iptables -L -n

Step 16: Verify Traffic Control

From Windows Client 2:
Ping
ping 192.168.56.10
Expected Result:
Request timed out.

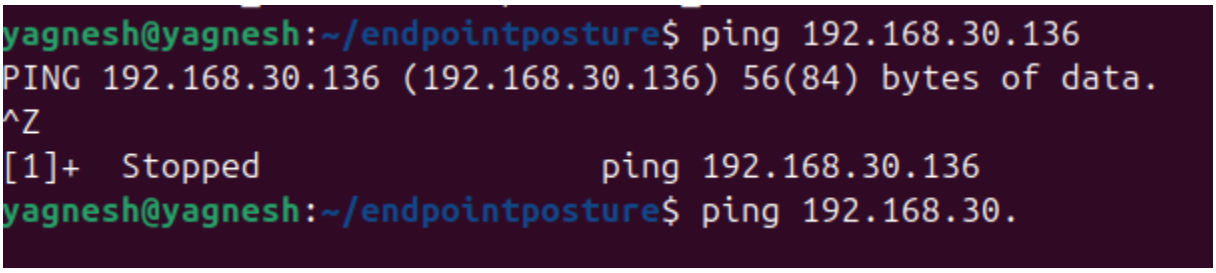

Fig 2.8 Ping is block

Step 17: SSH connection is also block

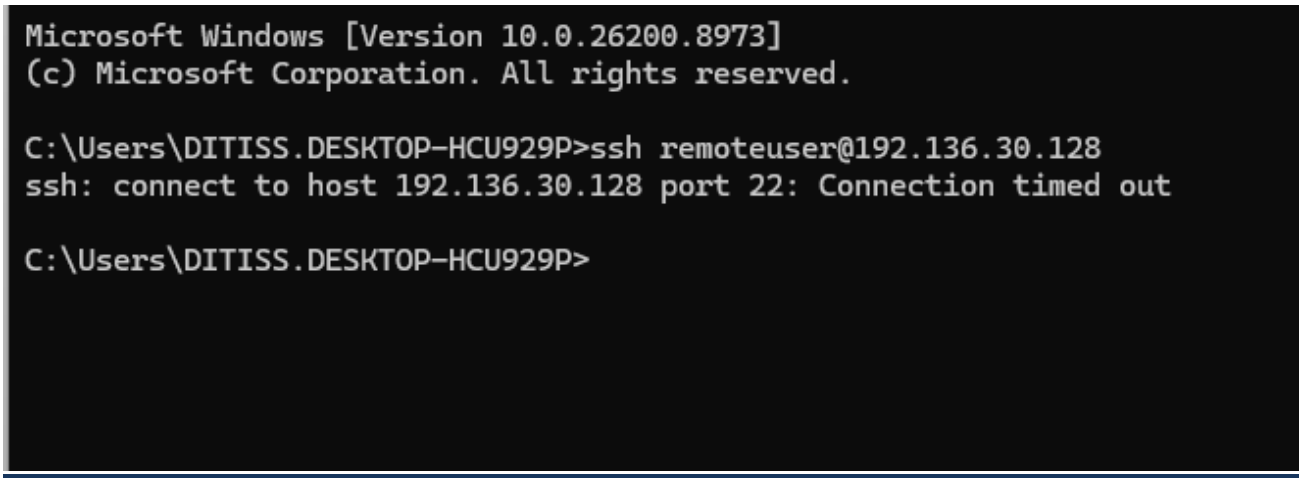

Fig : 2.9 SSH is Block

Step 18: Verify SSH Logs

Display SSH authentication logs.
sudo journalctl -u ssh
The log displays successful and failed login attempts.

Fig : 2.10 Logs

Expected Output

- Secure remote access using SSH.
- Firewall-based traffic control using iptables.
- Access allowed for authorized clients.
- Access denied for blocked clients.
- Verification through ping, SSH, and HTTP tests.

Learning Outcomes

- Understood the architecture and working of a Remote Access Gateway in a secure network environment.
- Configured an Ubuntu Server as a secure remote access gateway using the OpenSSH service.
- Created and managed authorized user accounts for secure remote access.
- Configured and verified SSH-based remote login from Windows client machines.
- Implemented network traffic control using **iptables** firewall rules.
- Allowed only authorized services (SSH, HTTP, and ICMP) while blocking unauthorized traffic.
- Restricted network access for specific client systems based on IP address.
- Verified firewall policies through practical testing using Ping, SSH, and HTTP services.
- Monitored remote access activities and network traffic using SSH logs and packet capture tools.
- Gained practical knowledge of access control, network security, and firewall management used in enterprise remote access environments.

Notes/Observations

Notes

- The Ubuntu Server was configured as the Remote Access Gateway.
- OpenSSH Server was installed and enabled to provide secure remote login services.
- A dedicated user account (remoteuser) was created for authenticated remote access.
- SSH configuration was modified to disable root login and allow only authorized users.
- The iptables firewall was configured to permit only required network services such as SSH (Port 22), HTTP (Port 80), and ICMP (Ping).
- Unauthorized network traffic was blocked using firewall rules.
- Two Windows virtual machines were used to simulate authorized and unauthorized remote clients.
- The Host-Only network was used for secure communication between the virtual machines, while the NAT adapter provided internet connectivity.

Observation

- The Remote Access Gateway successfully authenticated authorized users, restricted unauthorized access, and controlled incoming network traffic using firewall rules. The implementation demonstrated secure remote administration and practical traffic filtering techniques commonly used in enterprise network security environments.